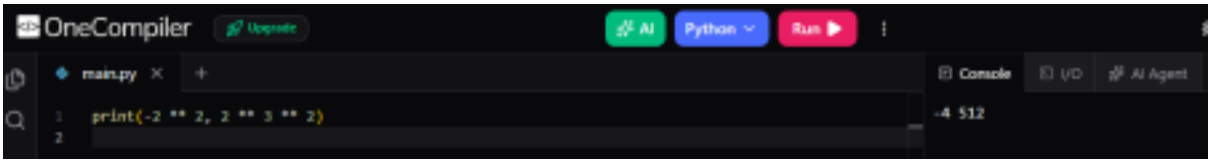
Resumo de Expressões, Atribuição e Controle

Aula 07 · Paradigmas de Linguagens de Programação

Quiz de abertura:

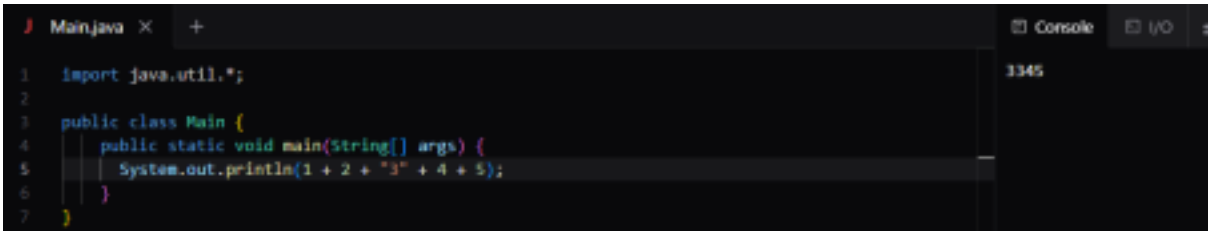
1.

Palpite antes de executar: 36



2.

Palpite antes de executar: 12



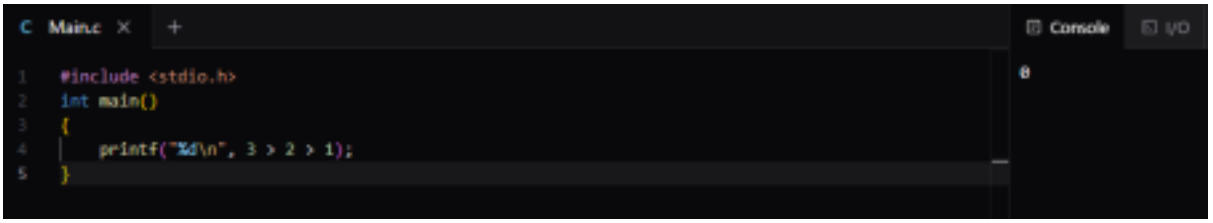
3.

Palpite antes de executar: “5”



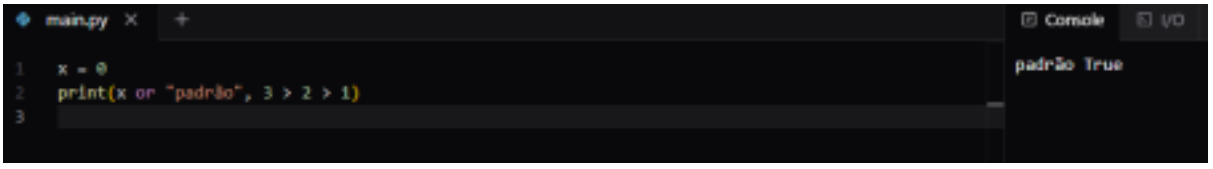
4.

Palpite antes de executar: 6



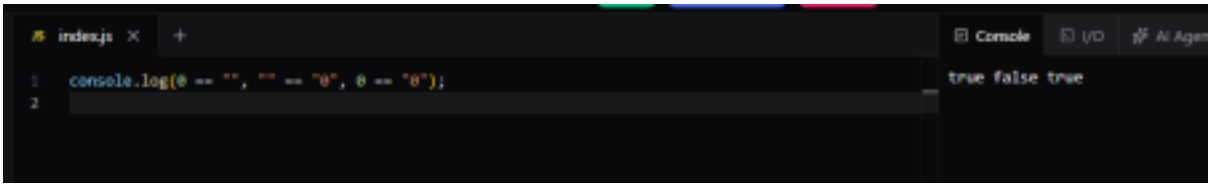
5.

Palpite antes de executar: 0 ou valor de x



6.

Palpite antes de executar: true false true



Expressões aritméticas: questões de projeto

Quais são as regras de precedência?

As regras de precedência definem a ordem de avaliação de operadores com **níveis de prioridade diferentes** em uma mesma expressão.

- **Projeto comum:** As linguagens adotam a hierarquia da matemática tradicional (exponenciação primeiro, depois multiplicação/divisão, e por fim adição/subtração).
- **Exceções de projeto:** Algumas linguagens quebram essa convenção. Em **APL**, por exemplo, não existe hierarquia de precedência; todos os operadores têm a mesma prioridade. Em **Smalltalk**, os operadores são tratados apenas como chamadas de métodos e são avaliados estritamente na ordem em que aparecem.

Quais são as regras de associatividade?

As regras de associatividade definem a ordem de avaliação de operadores que possuem o **mesmo nível de precedência** quando aparecem em sequência.

- **Associatividade à esquerda (Left-to-right):** É a regra para quase todos os operadores binários básicos (`+`, `-`, `*`, `/`). Uma expressão como `A - B + C` é tratada como `(A - B) + C`.
- **Associatividade à direita (Right-to-left):** Comumente aplicada a operadores unários (como o menos negativo `-A`) e ao operador de exponenciação (`**`). Em Fortran ou Python, `2 ** 3 ** 2` é avaliado de trás para frente: `2 ** (3 ** 2)`.

Em que ordem os operandos são avaliados?

Atenção para não confundir: a precedência dita a ordem dos *operadores*, mas o designer da linguagem também precisa decidir a ordem em que os *valores (operandos)* são lidos da memória ou calculados. Na expressão `(A + B) * (C + D)`, qual parênteses é processado primeiro?

- **Esquerda para a direita:** A maioria das linguagens modernas (como **Java** e

C#) força a avaliação dos operandos estritamente da esquerda para a direita. ● **Indefinida:** A linguagem **C/C++** deixa essa ordem a critério do compilador para permitir otimizações de código de acordo com a arquitetura do processador. Isso significa que o comportamento pode mudar dependendo da máquina onde o código roda.

Há restrições a efeitos colaterais?

Um efeito colateral (side effect) ocorre quando a avaliação de um operando (como uma chamada de função) modifica uma variável que faz parte do restante da expressão.

- **Linguagens Imperativas (C, C++, Java):** Geralmente não proíbem efeitos colaterais, o que pode gerar códigos imprevisíveis se o programador não for cuidadoso (ex: `A + funçao(A)` onde a função altera `A`).
- **Linguagens Funcionais Puras (Haskell, Lisp):** Proíbem estritamente efeitos colaterais. As funções não têm estado e não alteram variáveis globais. Isso elimina completamente o problema de previsibilidade na ordem de avaliação.

O usuário pode sobrecarregar operadores?

A sobrecarga de operadores permite que o programador redefina o significado de um operador (como `+` ou `*`) para funcionar com novos tipos de dados criados pelo usuário (como objetos ou structs).

- **Permitem sobrecarga: C++**, **C#**, **Python** e **Ruby** permitem que você redefina operadores para somar matrizes, concatenar objetos de forma customizada, etc.
- **Não permitem sobrecarga: Java** optou por não permitir a sobrecarga feita pelo usuário para manter o código mais simples e evitar abusos que dificultem a leitura (embora a linguagem use sobrecarga interna, como o `+` para strings).

Que mistura de tipos é permitida?

Esta questão lida com as regras de **expressões mistas** (mixed-mode expressions), onde operandos de tipos diferentes aparecem juntos (ex: somar um `int` com um `float`).

- **Coerção Implícita (Permissiva):** Linguagens como **C/C++** realizam conversões automáticas ampliando o tipo menor para o maior (ex: `int` vira `double` automaticamente antes da soma).

- **Rejeição Estrita (Segura):** Linguagens com sistemas de tipos muito rígidos, como **F#** ou **Go**, não permitem nenhuma mistura sem uma conversão explícita (type casting) feita pelo programador. Em Go, tentar fazer `inteiro + flutuante` resulta em um erro de compilação.

Estruturas de controle: quantas são necessárias?

Laços controlados por contador

C - o for é um laço lógico disfarçado

```c
for (int i = 0; i < 10; i++) {
    printf("%d ", i);
    i += 2;          // altera a contagem
}                    // 0 3 6 9

for (; i <= 100; )  // expressões opcionais
    soma += i++;
```

Python - o for percorre um iterável

```python
for i in range(0, 10):
    print(i, end=" ")
    i += 2           # NÃO muda nada
                     # 0 1 2 3 4 5 6 7 8 9

for i in range(10, 0, -3):   # 10 7 4 1
    ...
```

Questões de projeto (p. 342)
Qual é o tipo e o escopo da variável? Posso alterá-la no corpo? Os parâmetros são avaliados uma vez ou a cada volta?

Exercício em duplas: encontre o bug (1/3)

Para cada trecho: (a) qual é a saída? (b) qual regra causa o problema? (c) como corrigir?

1 · JavaScript

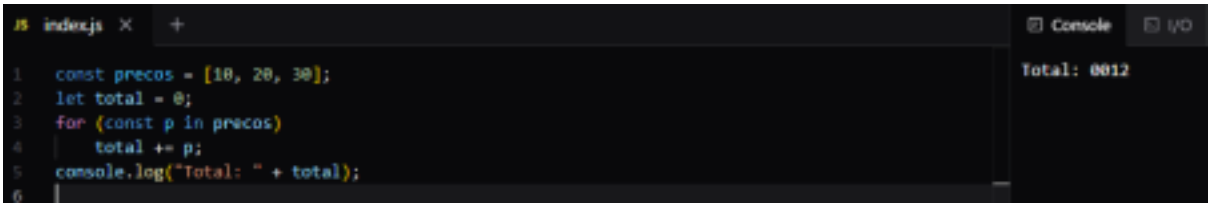
```javascript
const precos = [10, 20, 30];
let total = 0;
for (const p in precos)
    total += p;
console.log("Total: " + total);
```

2 · Python

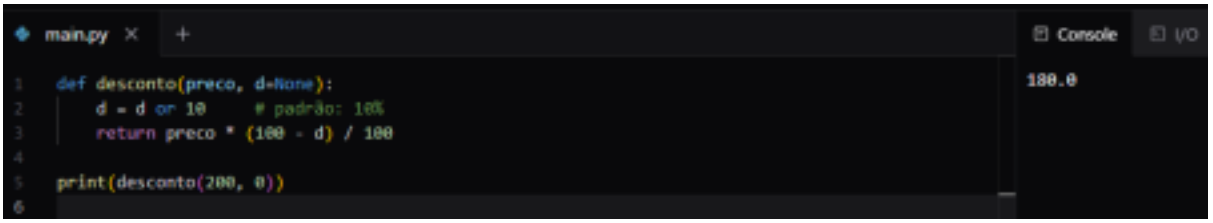
```python
def desconto(preco, d=None):
    d = d or 10      # padrão: 10%
    return preco * (100 - d) / 100

print(desconto(200, 0))
```

a:
JS:



PY:



b:
JS: total += p;
PY: d = d or 10

c:
JS: deveria ser total += precos[p];
PY: d = 10 if d is None else d

Exercício: encontre o bug (2/3)

Para cada trecho: (a) qual é a saída? (b) qual regra causa o problema? (c) como corrigir?

3 · Java

```java
int dia = 2; String nome = "";
switch (dia) {
  case 1: nome = "domingo";
  case 2: nome = "segunda";
  case 3: nome = "terça";
  default: nome = "inválido";
}
System.out.println(nome);
```

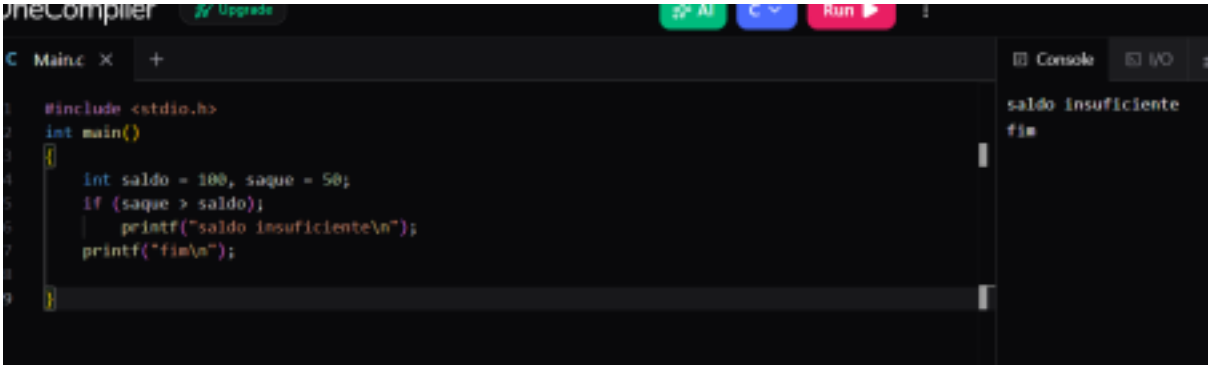
4 · C

```c
int saldo = 100, saque = 50;
if (saque > saldo);
    printf("saldo insuficiente\n");
printf("fim\n");
```

a:
JAVA:



C:



b:
JAVA: faltam os break no switch.
C: existe um “;” logo depois do if.

c:
JAVA:

```
 switch (dia) {
    case 1:
        nome = "domingo";
        break;
    case 2:
        nome = "segunda";
        break;
    case 3:
        nome = "terça";
        break;
    default:
        nome = "inválido";
 }
```

C:

```
if (saque > saldo)
    printf("saldo insuficiente\n");
```

Exercício: encontre o bug (3/3)

Para cada trecho: (a) qual é a saída? (b) qual regra causa o problema? (c) como corrigir?

5 - Go

```
x := 10
if x > 5 {
    x := x * 2
    fmt.Println("dentro:", x)
}
fmt.Println("fora:", x)
```

6 - Java

```
int acertos = 7, total = 10;
double taxa = acertos / total * 100;
System.out.println(taxa + "%");
```

a:
GO:

main.go

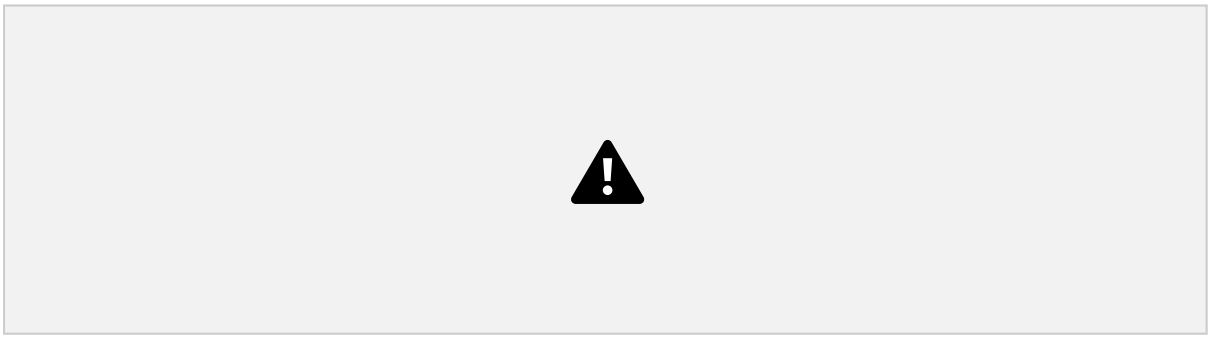
```go
package main
import "fmt"

func main() {
    x := 10
    if x > 5 {
        x := x * 2
        fmt.Println("dentro:", x)
    }
    fmt.Println("fora:", x)
}
```

Console

```
dentro: 20
fora: 10
```

JAVA:



b:
GO: dentro do if, foi usado “:=”
JAVA: acertos e total são int

c:
GO: x = x * 2
JAVA: double taxa = (double) acertos / total * 100;